their multiple outputs. In comparison, the nodes of the hidden and output layer are **active**. This means they modify the data as shown in Fig. 26-6. The variables: $X1_1, X1_2 \cdots X1_{15}$ hold the data to be evaluated (see Fig. 26-5). For example, they may be pixel values from an image, samples from an audio signal, stock market prices on successive days, etc. They may also be the output of some other algorithm, such as the classifiers in our cancer detection example: diameter, brightness, edge sharpness, etc.

Each value from the input layer is duplicated and sent to *all* of the hidden nodes. This is called a **fully interconnected** structure. As shown in Fig. 26-6, the values entering a hidden node are multiplied by **weights**, a set of predetermined numbers stored in the program. The weighted inputs are then added to produce a single number. This is shown in the diagram by the symbol, Σ. Before leaving the node, this number is passed through a nonlinear mathematical function called a *sigmoid*. This is an "s" shaped curve that limits the node's output. That is, the input to the sigmoid is a value between $-\infty$ and $+\infty$, while its output can only be between 0 and 1.

The outputs from the hidden layer are represented in the flow diagram (Fig 26-5) by the variables: $X2_1, X2_2, X2_3$ and $X2_4$. Just as before, each of these values is duplicated and applied to the next layer. The active nodes of the output layer combine and modify the data to produce the two output values of this network, $X3_1$ and $X3_2$.

Neural networks can have any number of layers, and any number of nodes per layer. Most applications use the three layer structure with a maximum of a few hundred input nodes. The hidden layer is usually about 10% the size of the input layer. In the case of target detection, the output layer only needs a single node. The output of this node is thresholded to provide a positive or negative indication of the target's presence or absence in the input data.

Table 26-1 is a program to carry out the flow diagram of Fig. 26-5. The key point is that this architecture is very simple and very generalized. This same flow diagram can be used for many problems, regardless of their particular quirks. The ability of the neural network to provide useful data manipulation lies in the proper selection of the *weights*. This is a dramatic departure from conventional information processing where solutions are described in step-by-step procedures.

As an example, imagine a neural network for recognizing objects in a sonar signal. Suppose that 1000 samples from the signal are stored in a computer. How does the computer determine if these data represent a submarine, whale, undersea mountain, or nothing at all? Conventional DSP would approach this problem with mathematics and algorithms, such as correlation and frequency spectrum analysis. With a neural network, the 1000 samples are simply fed into the input layer, resulting in values popping from the output layer. By selecting the proper weights, the output can be configured to report a wide range of information. For instance, there might be outputs for: submarine (yes/no), whale (yes/no), undersea mountain (yes/no), etc.